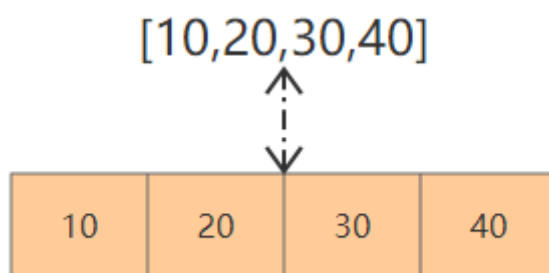


序列

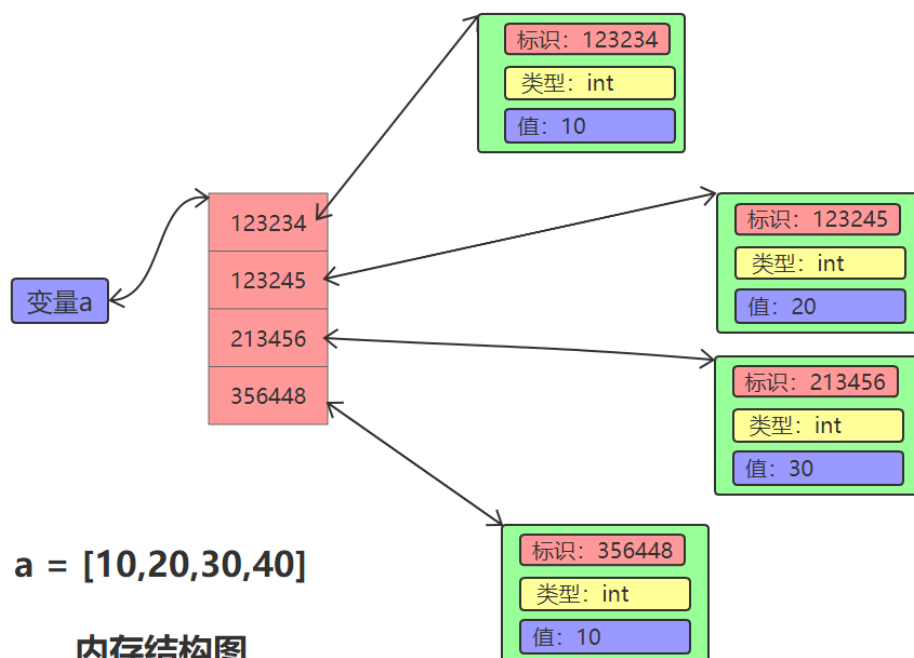


序列的本质和内存结构

序列是一种数据存储方式，用来存储一系列的数据。在内存中，序列就是一块用来存放多个值的连续的内存空间。比如一个整数序列 [10,20,30,40]，示意表示：



由于Python3中一切皆对象，在内存中实际是按照如下方式存储的：



从图示中，我们可以看出序列中存储的是整数对象的地址，而不是整数对象的值。

♡我们上一章学习的字符串就是一种序列。关于字符串里面很多操作，在这一章中仍然会用到，大家一定会感觉非常熟悉。

⚠本章内容，我们必须非常熟悉。无论是在学习还是工作中，序列都是每天都会用到的技术，可以非常方便的帮助我们进行数据存储的操作。

列表简介

- ① 列表：用于存储任意数目、任意类型的数据集合。
- ② 列表是内置可变序列，是包含多个元素的有序连续的内存空间。
列表的标准语法格式：

```
a = [10,20,30,40]
```

其中，10,20,30,40这些称为：列表a的**元素**。

- ③ 列表中的元素可以各不相同，可以是任意类型。比如：

```
a = [10,20,'abc',True]
```

- ④ Python的列表大小可变，根据需要随时增加或缩小。

列表对象的常用方法汇总如下，方便大家学习和查阅。

方法	要点	描述
list.append(x)	增加元素	将元素x增加到列表list尾部
list.extend(aList)	增加元素	将列表alist所有元素加到列表list尾部
list.insert(index,x)	增加元素	在列表list指定位置index处插入元素x
list.remove(x)	删除元素	在列表list中删除首次出现的指定元素x
list.pop([index])	删除元素	删除并返回列表list指定为止index处的元素，默认是最后一个元素
list.clear()	删除所有元素	删除列表所有元素，并不是删除列表对象
list.index(x)	访问元素	返回第一个x的索引位置，若不存在x元素抛出异常
list.count(x)	计数	返回指定元素x在列表list中出现的次数
len(list)	列表长度	返回列表中包含元素的个数
list.reverse()	翻转列表	所有元素原地翻转
list.sort()	排序	所有元素原地排序
list.copy()	浅拷贝	返回列表对象的浅拷贝

字符串和列表都是序列类型，一个字符串是一个字符序列，一个列表是任何元素的序列。我们前面学习的很多字符串的方法，在列表中也有类似的用法，几乎一模一样。

实时效果反馈

1. python中，关于序列说法错误的是：

- A 字符串和列表都是序列类型
- B 列表中的元素可以各不相同，但类型必须一致
- C 列表：用于存储任意数目、任意类型的数据集合
- D 内存中，序列就是一块用来存放多个值的连续的内存空间

答案

1=>B

列表的创建

基本语法 `[]` 创建

```
1 a = [10,20,'gaoqi','sxt']
2 b = []    #创建一个空的列表对象
3 print(a)
```

list()创建

使用list()可以将任何可迭代的数据转化成列表。

```
1 a = list()    #创建一个空的列表对象
2 b = list(range(10))    #结果: [0, 1, 2, 3, 4,
    5, 6, 7, 8, 9]
3 c = list("gaoqi,sxt") #结果: ['g', 'a', 'o',
    'q', 'i', ',', 's', 'x', 't']
```

range()创建整数列表

range()可以帮助我们非常方便的创建整数列表，这在开发中及其有用。语法格式为：

```
range([start,] end [,step])
```

start参数：可选，表示起始数字。默认是0

end参数：必选，表示结尾数字。

step参数: 可选, 表示步长, 默认为1

⚠ python3中range()返回的是一个range对象, 而不是列表。
我们需要通过list()方法将其转换成列表对象。

典型示例如下:

```
1 a = list(range(3,15,2)) #结果: [3, 5, 7, 9, 11, 13]
2 b = list(range(15,9,-1)) #结果: [15, 14, 13, 12, 11, 10]
3 c = list(range(3,-4,-1)) #结果: [3, 2, 1, 0, -1, -2, -3]
4 print(a,b,c)
```

推导式生成列表(简介一下, 重点在for循环后讲)

使用列表推导式可以非常方便的创建列表, 在开发中经常使用。

⚠但是, 由于涉及到for循环和if语句。在此, 仅做基本介绍。
在我们控制语句后面, 会详细讲解更多列表推导式的细节。

```
1 #循环创建多个元素 [0, 2, 4, 6, 8]
2 a = [x*2 for x in range(5)]
3
4 #通过if过滤元素[0, 18, 36, 54, 72, 90, 108, 126, 144, 162, 180, 198]
5 b = [x*2 for x in range(100) if x%9==0]
6
7 print(a,b)
```

实时效果反馈

1. 要创建列表[10,20,30,40,50,60,70,80,90]，如下代码错误的是：

A `a = [10,20,30,40,50,60,70,80,90]`

B `b = [x*10 for x in range(1,10)]`

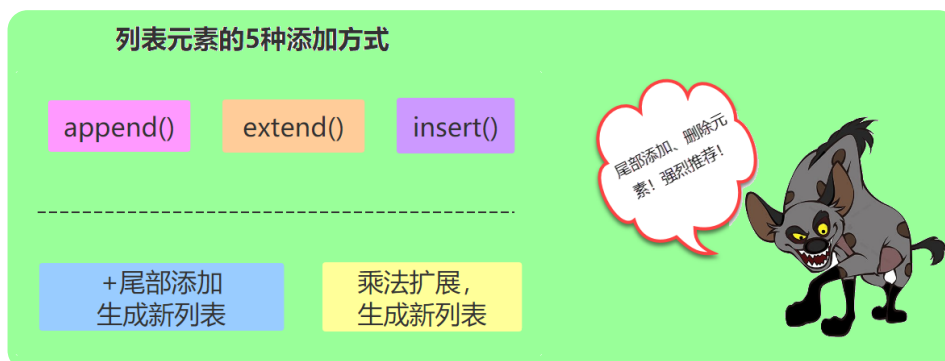
C `c = [x*10 for x in range(10) if x>0]`

D `d = list(range(10)*10)`

答案

1=>D

列表元素的增加



当列表增加和删除元素时，列表会自动进行内存管理，大大减少了程序员的负担。但这个特点涉及列表元素的大量移动，效率较低。

⚠ 除非必要，我们一般只在列表的尾部添加元素或删除元素，这会大大提高列表的操作效率。

append()方法

原地修改列表对象，是真正的列表尾部添加新的元素，**速度最快，推荐使用**。

```
1 a = [20,40]
2 a.append(80)
3 print(a)           #结果: [20, 40, 80]
```

+运算符操作

并不是真正的尾部添加元素，而是创建新的列表对象；将原列表的元素和新列表的元素依次复制到新的列表对象中。这样，会涉及大量的复制操作，对于操作大量元素不建议使用。

```
1 a = [20,40]
2 print(id(a))
3 a = a+[50]
4 print(id(a))  #两次地址不一样，创建了新的对象
```

通过如上测试，我们发现变量a的地址发生了变化。也就是创建了新的列表对象。

extend()方法

将目标列表的所有元素添加到本列表的尾部，属于原地操作，不创建新的列表对象。

```
1 a = [20,40]
2 print(id(a))
3 b = [50,60]
4 a.extend(b)      #原对象修改
5 print(id(a))
6 a = a+b          #产生新对象
7 print(id(a))
```

insert()插入元素

使用 `insert()` 方法可以将指定的元素插入到列表对象的任意制定位置。这样会让插入位置后面所有的元素进行移动，会影响处理速度。涉及大量元素时，尽量避免使用。类似发生这种移动的函数还有：

`remove()`、`pop()`、`del()`，它们在删除**非尾部元素**时也会发生操作位置后面元素的移动。

```
1 a = [10,20,30]
2 a.insert(2,100)
3 print(a)      #结果: [10, 20, 100, 30]
```

乘法扩展

使用乘法扩展列表，生成一个新列表，新列表元素是原列表元素的多次重复。

```
1 a = ['sxt',100]
2 b = a*3
3 print(a) #结果: ['sxt', 100]
4 print(b) #结果: ['sxt', 100, 'sxt', 100, 'sxt', 100]
```

适用于乘法操作的，还有：字符串、元组。例如：

```
1 c = 'sxt'
2 d = c*3      #结果: 'sxtsxtsxt'
```

实时效果反馈

1. 在列表[10,20,30]后添加元素40，如下代码，正确并且速度最快的是：

A [10,20,30]+[40]

B [10,20,30]*40

C [10,20,30].append(40)

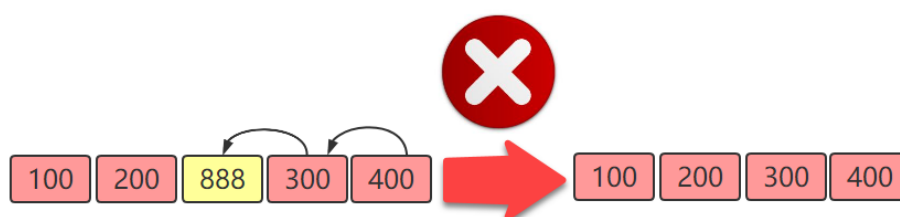
D [10,20,30].insert(2,40)

答案

1=>C

列表元素的删除

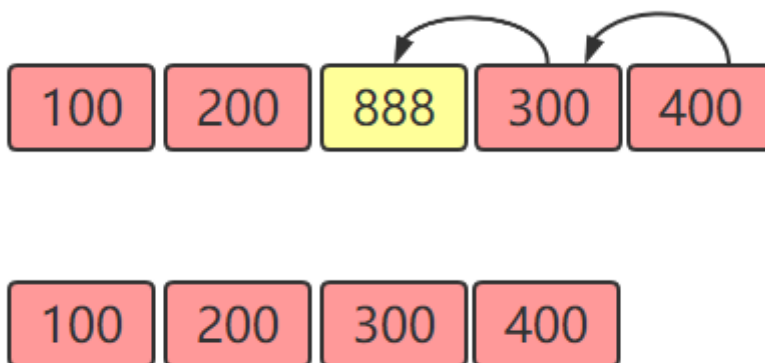
删除元素本质是：元素的拷贝



del 删除

删除列表指定位置的元素。

```
1 a = [100,200,888,300,400]
2 del a[2]
3 print(a)      #结果: [100,200,300,400]
```



pop()方法

pop()删除并返回指定位置元素，如果未指定位置则默认操作列表最后一个元素。

```
1 a = [10,20,30,40,50]
2 b1 = a.pop()      #结果: b1=50
3 print(a,b1)      #结果: [10, 20, 30, 40]  50
4 b2 = a.pop(1)
5 print(a,b2)      #结果: [10, 30, 40],20
```

remove()方法

删除首次出现的指定元素，若不存在该元素抛出异常。

```
1 a = [10,20,30,40,50,20,30,20,30]
2 a.remove(20)    #[10, 30, 40, 50, 20, 30, 20,
3                 30]
3 a.remove(100)   #报错:
                  ValueError:list.remove(x): x not in list
```

实时效果反馈

1. 删除 `a = [10,20,30,40,50]` 最后一个元素50，如下代码错误的是：

- A `a.pop()`
- B `a.remove(50)`
- C `del a[50]`
- D `a.pop(4)`

答案

1=>C

列表元素访问和计数

访问列表中的元素和计数

1. 索引直接访问元素
2. `index()` 获得指定元素
在列表中首次出现的索引
3. `count()` 获得指定元素
在列表中出现的次数
4. `in/not in`



通过索引直接访问元素

我们可以通过索引直接访问元素。索引的区间在 `[0, 列表长度-1]` 这个范围。超过这个范围则会抛出异常。

```
1 a = [10, 20, 30, 40, 50, 20, 30, 20, 30]
2 print(a[2])    #结果: 30
3 print(a[10])   #报错: IndexError: list index out of range
```

index()获得指定元素在列表中首次出现的索引

`index()` 可以获取指定元素首次出现的索引位置。语法是: `index(value, [start, [end]])`。其中, `start` 和 `end` 指定了搜索的范围。

```
1 >>> a = [10, 20, 30, 40, 50, 20, 30, 20, 30]
2 >>> a.index(20)    #结果: 1
3 >>> a.index(20, 3)  #结果: 5    从索引位置3开始往后搜索的第一个20
4 >>> a.index(30, 5, 7) #结果: 6  从索引位置5到7这个区间, 第一次出现30元素的位置
```

count()获得指定元素在列表中出现的次数

`count()` 可以返回指定元素在列表中出现的次数。

```
1 >>> a = [10, 20, 30, 40, 50, 20, 30, 20, 30]
2 >>> a.count(20)
3 3
```

len()返回列表长度

len()返回列表长度，即列表中包含元素的个数。

```
1 >>> a = [10,20,30]
2 >>> len(a)
3 3
```

成员资格判断

判断列表中是否存在指定的元素，我们可以使用 `count()` 方法，返回0则表示不存在，返回大于0则表示存在。但是，一般我们会使用更加简洁的 `in` 关键字来判断，直接返回 `True` 或 `False`

```
1 >>> a = [10,20,30,40,50,20,30,20,30]
2 >>> 20 in a
3 True
4 >>> 100 not in a
5 True
6 >>> 30 not in a
7 False
```

实时效果反馈

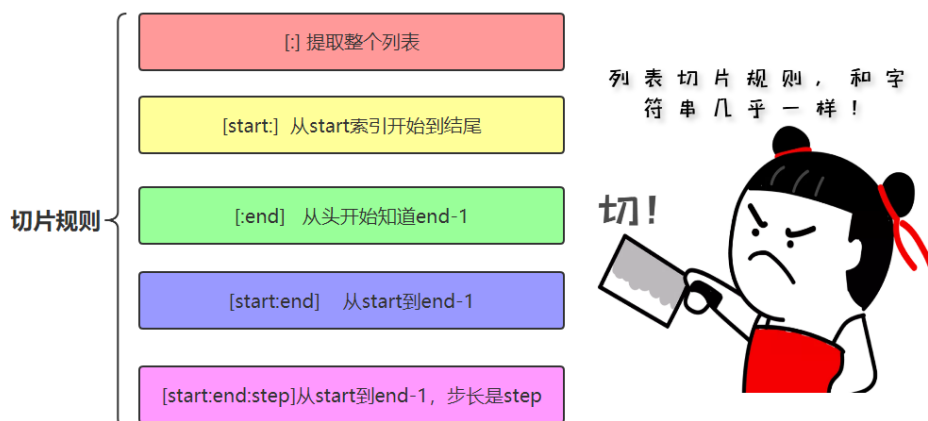
1. 判断 `a = [10,20,30,40,50,20,30,20,30]` 是否包含元素 `20`，如下代码错误的是：

- A `20 in a`
- B `0<=a.index(20)<=len(a)-1`
- C `a.count(20)==0`
- D `a[20] is not None`

答案

1=>D

切片操作



类似字符串的切片操作，对于列表的切片操作和字符串类似。

切片是Python序列及其重要的操作，适用于列表、元组、字符串等等。

切片slice操作可以让我们快速提取子列表或修改。标准格式为：

`[起始偏移量start:终止偏移量end[:步长step]]`

典型操作(三个量为正数的情况)如下：

操作和说明	示例	结果
<code>[]</code> 提取整个列表	<code>[10,20,30][:]</code>	<code>[10,20,30]</code>
<code>[start:]</code> 从start索引开始到结尾	<code>[10,20,30][1:]</code>	<code>[20,30]</code>
<code>[:end]</code> 从头开始知道end-1	<code>[10,20,30][:2]</code>	<code>[10,20]</code>
<code>[start:end]</code> 从start到end-1	<code>[10,20,30,40][1:3]</code>	<code>[20,30]</code>
<code>[start:end:step]</code> 从start提取到end-1，步长是step	<code>[10,20,30,40,50,60,70][1:6:2]</code>	<code>[20, 40, 60]</code>

其他操作（三个量为负数）的情况：

示例	说明	结果
<code>[10,20,30,40,50,60,70][-3:]</code>	倒数三个	<code>[50,60,70]</code>
<code>[10,20,30,40,50,60,70] [-5:-3]</code>	倒数第五个到倒数第三个(包头不包尾)	<code>[30,40]</code>
<code>[10,20,30,40,50,60,70][::-1]</code>	步长为负，从右到左反向提取	<code>[70, 60, 50, 40, 30, 20, 10]</code>

切片操作时，起始偏移量和终止偏移量不在 `[0,字符串长度-1]` 这个范围，也不会报错。起始偏移量 小于0 则会当做 0，终止偏移量 大于“长度-1” 会被当成 “长度-1”。例如：

`[10,20,30,40][1:30]`

结果： `[20, 30, 40]`

我们发现正常输出了结果，没有报错。

实时效果反馈

1. 如下代码，执行结果的是：

```
1 [10, 20, 30, 40][1:3]
```

A `[20, 30]`

B `[10, 20, 30]`

C `[20, 30, 40]`

D `[40]`

2. 如下代码，执行结果的是：

```
1 [10, 20, 30, 40, 50, 60, 70][::-1]
```

A [70]

B [50]

C [10]

D [70, 60, 50, 40, 30, 20, 10]

答案

1=>A 2=>D

列表的遍历

```
1 a = [10, 20, 30, 40]
2 for obj in a:           #obj是临时变量名称，随意起
3     print(obj)
```

复制列表所有的元素到新列表对象

如下代码实现列表元素的复制了吗？

```
1 list1 = [30, 40, 50]
2 list2 = list1
```

只是将list2也指向了列表对象，也就是说list2和list1持有地址值是相同的，列表对象本身的元素并没有复制。

我们可以通过如下简单方式，实现列表元素内容的复制：

```
1 list1 = [30,40,50]
2 list2 = [] + list1  #生成了新列表对象
```

注：我们后面也会学习copy模块，使用浅复制或深复制实现我们的复制操作。

列表排序

修改原列表，不建新列表的排序

```
1 >>> a = [20,10,30,40]
2 >>> id(a)
3 46017416
4 >>> a.sort()          #默认是升序排列
5 >>> a
6 [10, 20, 30, 40]
7 >>> a = [10,20,30,40]
8 >>> a.sort(reverse=True)  #降序排列
9 >>> a
10 [40, 30, 20, 10]
11 >>> import random
12 >>> random.shuffle(a)  #打乱顺序
13 >>> a
14 [20, 40, 30, 10]
```

建新列表的排序

我们也可以通过内置函数sorted()进行排序，这个方法返回新列表，不对原列表做修改。

```
1 >>> a = [20, 10, 30, 40]
2 >>> id(a)
3 46016008
4 >>> b = sorted(a)           #默认升序
5 >>> b
6 [10, 20, 30, 40]
7 >>> id(b)
8 45907848
9 >>> c = sorted(a, reverse=True)   #降序
10 >>> c
11 [40, 30, 20, 10]
```

通过上面操作，我们可以看出，生成的列表对象b和c都是完全新的列表对象。

reversed()返回迭代器

内置函数reversed()也支持进行逆序排列，与列表对象reverse()方法不同的是，内置函数reversed()不对原列表做任何修改，只是返回一个逆序排列的迭代器对象。

```
1 >>> a = [20, 10, 30, 40]
2 >>> c = reversed(a)
3 >>> c
4 <list_reverseiterator object at
  0x0000000002BCCEB8>
5 >>> list(c)
6 [40, 30, 10, 20]
7 >>> list(c)
8 []
```

我们打印输出c发现提示是：list_reverseiterator。也就是一个迭代对象。同时，我们使用list(c)进行输出，发现只能使用一次。第一次输出了元素，第二次为空。那是因为迭代对象在第一次时已经遍历结束了，第二次不能再使用。

⚠ 关于迭代对象的使用，后续章节会进行详细讲解。

列表相关的其他内置函数汇总

max和min

用于返回列表中最大和最小值。

```
1 >>> a = [3,10,20,15,9]
2 >>> max(a)
3 20
4 >>> min(a)
5 3
```

sum

对数值型列表的所有元素进行求和操作，对非数值型列表运算则会报错。

```
1 >>> a = [3,10,20,15,9]
2 >>> sum(a)
3 57
```

实时效果反馈

1. 如下代码，说法正确的是：

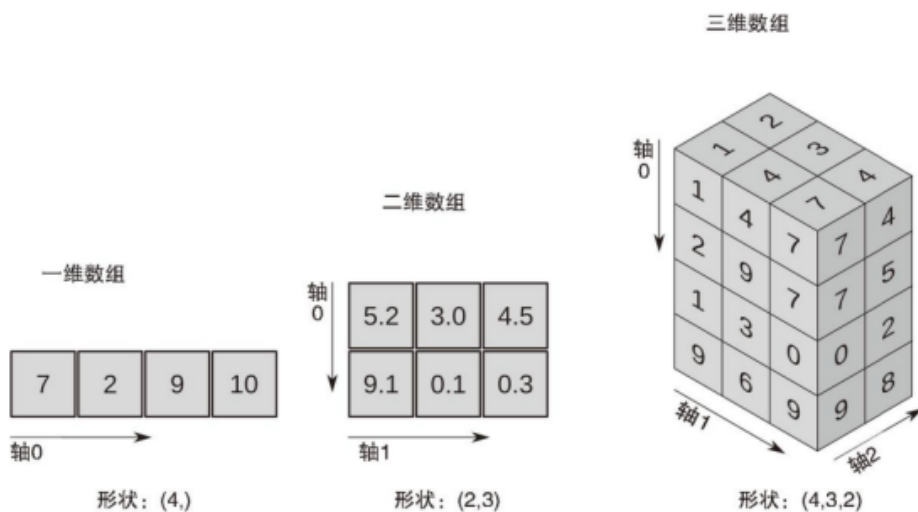
```
1 list1 = [30,40,50]
2 list2 = [] + list1
```

- A** list2是一个新的列表对象
- B** list2和list1是同一个对象
- C** list2不是一个新对象
- D** list2是一个空的列表对象

答案

1=>A

多维列表



二维列表

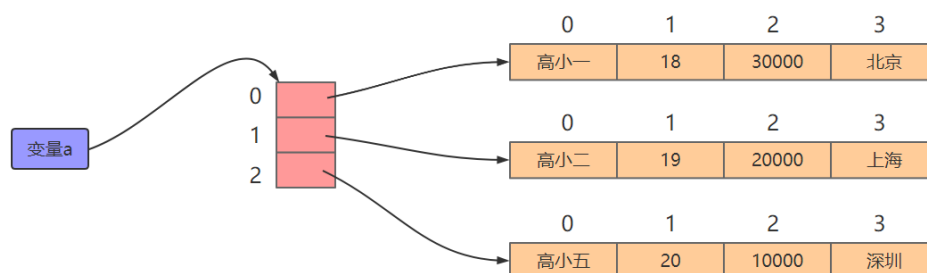
- 一维列表可以帮助我们存储一维、线性的数据。
- 二维列表可以帮助我们存储二维、表格的数据。例如下表的数据：

姓名	年龄	薪资	城市
高小一	18	30000	北京
高小二	19	20000	上海
高小五	20	10000	深圳

源码：

```
1 a = [  
2     ["高小一", 18, 30000, "北京"],  
3     ["高小二", 19, 20000, "上海"],  
4     ["高小五", 20, 10000, "深圳"],  
5 ]
```

内存结构图：



```
1 >>> print(a[1][0],a[1][1],a[1][2])
2 高小二 19 20000
```

嵌套循环打印二维列表所有的数据（由于没有学循环，照着敲一遍即可）：

```
1 a = [
2     ["高小一",18,30000,"北京"],
3     ["高小二",19,20000,"上海"],
4     ["高小一",20,10000,"深圳"],
5 ]
6 for m in range(3):
7     for n in range(4):
8         print(a[m][n],end="\t")
9     print() #打印完一行，换行
```

运行结果：

高小一 18 30000 北京

高小二 19 20000 上海

高小一 20 10000 深圳

实时效果反馈

1. 如下列表，想访问到"深圳"，正确的是：

```

1 a = [
2     ["高小一", 18, 30000, "北京"],
3     ["高小二", 19, 20000, "上海"],
4     ["高小一", 20, 10000, "深圳"]
5 ]

```

A `a[1][3]`

B `a[2][3]`

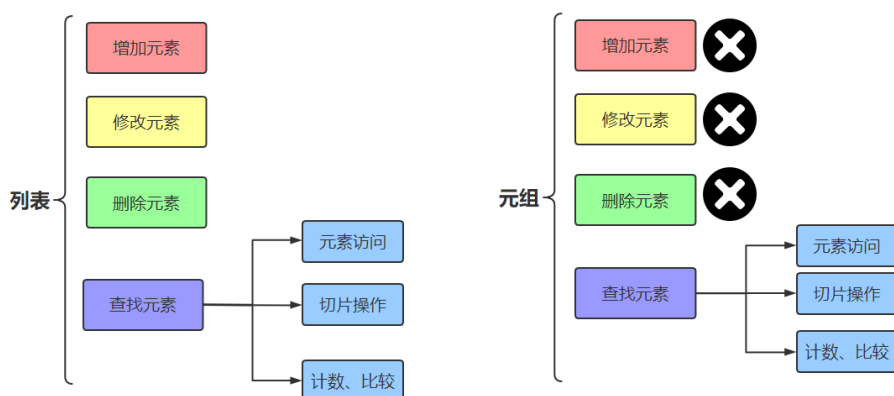
C `a[3][3]`

D `a[3][2]`

答案

1=>B

元组tuple



- 列表属于可变序列，可以任意修改列表中的元素。
- 元组属于不可变序列，不能修改元组中的元素。

因此，元组没有增加元素、修改元素、删除元素相关的方法。

因此，我们只需学元组的创建和删除，元素的访问和计数即可。元组支持如下操作：

- ① 索引访问
- ② 切片操作
- ③ 连接操作
- ④ 成员关系操作
- ⑤ 比较运算操作
- ⑥ 计数：元组长度len()、最大值max()、最小值min()、求和sum()等

元组的创建

- ① 通过()创建元组。小括号可以省略。

`a = (10,20,30)` 或者 `a = 10,20,30`

如果元组只有一个元素，则必须后面加逗号。这是因为解释器会把(1)解释为整数1，(1,)解释为元组。

`a = 10,`

```

1 >>> a = (1)
2 >>> type(a)
3 <class 'int'>
4 >>> a = (1,)      #或者 a = 1,
5 >>> type(a)
6 <class 'tuple'>

```

- ② 通过tuple()创建元组

`tuple(可迭代的对象)`

```

1 a = tuple()      #创建一个空元组对象
2 b = tuple("abc")
3 c = tuple(range(3))
4 d = tuple([2,3,4])

```

总结：

- ① tuple()可以接收列表、字符串、其他序列类型、迭代器等生成元组。

2 list()可以接收元组、字符串、其他序列类型、迭代器等生成列表。

实时效果反馈

1. 关于元组，如下说法错误的是：

- A 元组属于不可变序列，不能修改元组中的元素。
- B 元组没有增加元素、修改元素、删除元素相关的方法
- C `a = (1)` 定义的是一个元组
- D `a = (1,)` 定义的是一个元组

答案

1=>C

元组的元素访问和计数

① 元组的元素不能修改

```
1 >>> a = (20,10,30,9,8)
2 >>> a[3]=33
3 Traceback (most recent call last):
4   File "<pyshell#313>", line 1, in
    <module>
5     a[3]=33
6 TypeError: 'tuple' object does not support
    item assignment
```

- ② 元组的元素访问、index()、count()、切片等操作，和列表一样。

```

1 >>> a = (20,10,30,9,8)
2 >>> a[1]
3 10
4 >>> a[1:3]
5 (10, 30)
6 >>> a[:4]
7 (20, 10, 30, 9)

```

- ③ 列表关于排序的方法`list.sorted()`是修改原列表对象，元组没有该方法。如果要对元组排序，只能使用内置函数`sorted(tupleObj)`，并生成新的列表对象。

```

1 a = (20,10,30,9,8)
2 b = sorted(a)    #b是新对象，内容是：[8, 9,
                  10, 20, 30]

```

zip

`zip(列表1, 列表2, ...)`将多个列表对应位置的元素组合成为元组，并返回这个zip对象。

- 如果各个迭代器的元素个数不一致，则返回列表长度与最短的对象相同

```

1 a = [10,20,30]
2 b = [40,50,60]
3 c = [70,80,90,100]
4 d = zip(a,b,c)
5 print(d)    #zip object
6 e = list(d) #列表：[(10, 40, 70), (20, 50,
                  80), (30, 60, 90)]
7 print(e)

```

实时效果反馈

1. 如下关于元组 `a = (20,10,30,9,8)` 的代码，会报错的是：

A `print(a[1:3])`

B `b = sorted(a)`

C `a[3]=33`

D `b = list(a)`

答案

1=>C

生成器推导式创建元组

- ① 从形式上看，生成器推导式与列表推导式类似，只是生成器推导式使用小括号。
- ② 列表推导式直接生成列表对象，生成器推导式生成的不是列表也不是元组，而是一个生成器对象。
- ③ 我们可以通过生成器对象，转化成列表或者元组。也可以使用生成器对象的 `__next__()` 方法进行遍历，或者直接作为迭代器对象来使用。不管什么方式使用，元素访问结束后，如果需要重新访问其中的元素，必须重新创建该生成器对象。

【操作】生成器的使用测试

```

1  #列表推导式： [0, 2, 4, 6, 8]
2  #a = [x*2 for x in range(5)]
3  #print(a)
4
5  s = (x*2 for x in range(5))
6  print(s)    #<generator object <genexpr> at
               0x0000021C80BE2880>
7  b = tuple(s)
8  print(b)    #(0, 2, 4, 6, 8)
9  c = tuple(s)
10 print(c)    #()
11

```

```
12 s2 = (x for x in range(3))
13 print(s2.__next__())      #0
14 print(s2.__next__())      #1
15 print(s2.__next__())      #2
16 print(s2.__next__())      #报错:
                             StopIteration
```

元组总结

- 1 元组的核心特点是：不可变序列。
- 2 元组的访问和处理速度比列表快。
- 3 与整数和字符串一样，元组可以作为字典的键，列表则永远不能作为字典的键使用。

实时效果反馈

1. 如下代码，如下说法正确的是：

```
1 s = (x*2 for x in range(5))
```

- ☐ A s是一个元组
- ☐ B s是一个列表
- ☒ C s是一个生成器对象，只能被迭代一次
- ☐ D s是一个生成器对象，可以被反复迭代多次

答案

1=>C

字典

字典是“键值对”的无序可变序列，字典中的每个元素都是一个“键值对”，包含：“键对象”和“值对象”。可以通过“键对象”实现快速获取、删除、更新对应的“值对象”。



一个典型的字典的定义方式：

```
a = {'name':'gaoqi', 'age':18, 'job':'programmer'}
```

列表中我们通过“下标数字”找到对应的对象。字典中通过“键对象”找到对应的“值对象”。

- 1 “键”是任意的不可变数据，比如：整数、浮点数、字符串、元组。
- 2 但是：列表、字典、集合这些可变对象，不能作为“键”。
- 3 并且“键”不可重复。
- 4 “值”可以是任意的数据，并且可重复。

字典的创建

- 1 我们可以通过{}、dict()来创建字典对象。

```

1 a =
  {'name': 'gaoqi', 'age': 18, 'job': 'programmer'}
2 b =
  dict(name='gaoqi', age=18, job='programmer')
3 a = dict([("name", "gaoqi"), ("age", 18)])
4 c = {}    #空的字典对象
5 d = dict() #空的字典对象

```

② 通过zip()创建字典对象

```

1 k = ['name', 'age', 'job']
2 v = ['gaoqi', 18, 'teacher']
3 d = dict(zip(k, v))
4 print(d) #{'name': 'gaoqi', 'age': 18, 'job': 'teacher'}

```

③ 通过fromkeys创建值为空的字典

```

1 f = dict.fromkeys(['name', 'age', 'job'])
2 print(f) #结果: {'name': None, 'age': None, 'job': None}

```

实时效果反馈

1. 关于字典，如下说法错误的是：

- ☒ A 字典是“键值对”的有序可变序列
- ☐ B 字典中的“值对象”可以是任意类型
- ☐ C 列表对象不可以作为字典的“键对象”
- ☐ D 元组对象可以作为字典的“键对象”

答案

1=>A

字典元素的访问

为了测试各种访问方法，我们这里设定一个字典对象：

```
1 a =  
  {'name': 'gaoqi', 'age': 18, 'job': 'programmer'}
```

- ① 通过 [键] 获得“值”。若键不存在，则抛出异常。

```
1 a =  
  {'name': 'gaoqi', 'age': 18, 'job': 'programmer'  
  }  
2 b = a['name']  
3 print(b)
```

- ② 通过get()方法获得“值”。♡推荐使用。优点是：指定键不存在，返回None；也可以设定指定键不存在时默认返回的对象。推荐使用get()获取“值对象”

```
1 a =  
  {'name': 'gaoqi', 'age': 18, 'job': 'programmer'  
  }  
2 b = a.get('name')  
3 c = a.get('gender', '不存在')  
4 print(b)  
5 print(c)
```

- ③ 列出所有的键值对

```

1 a =
  {'name': 'gaoqi', 'age': 18, 'job': 'programmer'}
2 b = a.items()
3 print(b)  #dict_items([('name', 'gaoqi'),
  ('age', 18), ('job', 'programmer')])

```

④ 列出所有的键，列出所有的值

```

1 a =
  {'name': 'gaoqi', 'age': 18, 'job': 'programmer'}
2 k = a.keys()
3 v = a.values()
4 print(k)  #dict_keys(['name', 'age',
  'job'])
5 print(v)  #dict_values(['gaoqi', 18,
  'programmer'])

```

⑤ len() 键值对的个数

```

1 a =
  {'name': 'gaoqi', 'age': 18, 'job': 'programmer'}
2 num = len(a)
3 print(num)  #3

```

⑥ 检测一个“键”是否在字典中

```

1 a =
  {'name': 'gaoqi', 'age': 18, 'job': 'programmer'}
2 print("name" in a)  #True

```

实时效果反馈

1. 如下代码，如下说法正确的是：

```

1 a =
  {'name': 'gaoqi', 'age': 18, 'job': 'programmer'}
2 c = a.get('gender', '一个男人')

```

- A 会报错
- B 变量 `c` 是 `None`
- C 变量 `c` 是 一个男人
- D 变量 `c` 是 空字符串

答案

1=>C

字典元素添加、修改、删除

- ① 给字典新增“键值对”。如果“键”已经存在，则覆盖旧的键值对；如果“键”不存在，则新增“键值对”

```

1 a =
  {'name': 'gaoqi', 'age': 18, 'job': 'programmer'}
2 a['address'] = '西三旗1号院'
3 a['age'] = 16
4 print(a)
5 #{'name': 'gaoqi', 'age': 16, 'job':
  'programmer', 'address': '西三旗1号院'}

```

- ② 使用 `update()` 将新字典中所有键值对全部添加到旧字典对象上。如果 `key` 有重复，则直接覆盖

```

1 a =
  {'name': 'gaoqi', 'age': 18, 'job': 'programmer'
  }
2 b =
  {'name': 'gaoxixi', 'money': 1000, 'gender': '男
  的'}
3 a.update(b)
4 print(a)
5 #{'name': 'gaoxixi', 'age': 18, 'job':
  'programmer', 'money': 1000, 'gender': '男
  的'}

```

- ③ 字典中元素的删除，可以使用 `del()` 方法；或者 `clear()` 删除所有键值对；`pop()` 删除指定键值对，并返回对应的“值对象”

```

1 a =
  {'name': 'gaoqi', 'age': 18, 'job': 'programmer'
  }
2 del(a['name'])
3 print(a)          #{'age': 18, 'job':
  'programmer'}
4 age = a.pop('age')
5 print(age)        #18

```

- ④ `popitem()`：随机删除和返回该键值对。字典是“无序可变序列”，因此没有第一个元素、最后一个元素的概念；`popitem` 弹出随机的项，因为字典并没有“最后的元素”或者其他有关顺序的概念。若想一个接一个地移除并处理项，这个方法就非常有效（因为不用首先获取键的列表）

```
1 a =  
  {'name': 'gaoqi', 'age': 18, 'job': 'programmer'  
  }  
2 r1 = a.popitem()  
3 r2 = a.popitem()  
4 r3 = a.popitem()  
5 print(a)      #{} 
```

实时效果反馈

1. 如下代码，说法正确的是：

```
1 a =  
  {'name': 'gaoqi', 'age': 18, 'job': 'programmer'}  
2 a['address'] = '西三旗1号院'  
3 a['age'] = 16
```

- ☒ A 字典a中会有两个age键值对: `age:18, age:16`
- ☐ B 字典a中只有一个age: `age:18`
- ☐ C 字典a中只有一个age: `age:16`
- ☐ D 会报错。

答案

1=>C

序列解包

序列解包可以用于元组、列表、字典。序列解包可以让我们方便的对多个变量赋值。

```

1 x,y,z=(20,30,10)
2 (a,b,c)=(9,8,10)
3 [m,n,p]=[10,20,30]

```

序列解包用于字典时，默认是对“键”进行操作；如果需要对键值对操作，则需要使用items()；如果需要对“值”进行操作，则需要使用values()；

```

1 s = {'name':'gaoqi','age':18,'job':'teacher'}
2 name,age,job=s                #默认对键进行操作
3 print(name)                   #name
4 name,age,job=s.items()        #对键值对进行操作
5 print(name)                   #('name', 'gaoqi')
6 name,age,job=s.values()       #对值进行操作
7 print(name)                   #gaoqi

```

实时效果反馈

1. 如下代码，如下说法正确的是：

```

1 s = {'name':'gaoqi','age':18,'job':'teacher'}
2 a,b,c=s

```

- ☒ A 变量a的值是s
- ☐ B 变量a的值是字符串"name"
- ☐ C 变量a的值是字符串"gaoqi"
- ☐ D 会报错

答案

1=>B

表格数据使用字典和列表存储和访问

姓名	年龄	薪资	城市
高小一	18	30000	北京
高小二	19	20000	上海
高小五	20	10000	深圳

```
1  r1 = {"name": "高小
    一", "age": 18, "salary": 30000, "city": "北京"}
2  r2 = {"name": "高小
    二", "age": 19, "salary": 20000, "city": "上海"}
3  r3 = {"name": "高小
    五", "age": 20, "salary": 10000, "city": "深圳"}
4
5  tb = [r1, r2, r3]
6
7  #获得第二行的人的薪资
8  print(tb[1].get("salary"))
9
10 #打印表中所有的薪资
11 for i in range(len(tb)):    # i --> 0, 1, 2
12     print(tb[i].get("salary"))
13
14 #打印表的所有数据
15 for i in range(len(tb)):
16
17     print(tb[i].get("name"), tb[i].get("age"), tb
18           [i].get("salary"), tb[i].get("city"))
```

1. 如下代码，打印输出的是：

```
1 r1 = {"name": "高小  
一", "age": 18, "salary": 30000, "city": "北京"}  
2 r2 = {"name": "高小  
二", "age": 19, "salary": 20000, "city": "上海"}  
3 r3 = {"name": "高小  
五", "age": 20, "salary": 10000, "city": "深圳"}  
4  
5 tb = [r1, r2, r3]  
6 print(tb[2].get("city"))
```

A "city": "深圳"

B 上海

C 北京

D 深圳

答案

1=>D

字典核心底层原理(重要)

字典对象的核心是散列表。散列表是一个稀疏数组（总是有空白元素的数组），数组的每个单元叫做 `bucket`。每个 `bucket` 有两部分：一个是键对象的引用，一个是值对象的引用。

由于，所有 `bucket` 结构和大小一致，我们可以通过偏移量来读取指定 `bucket`。

0	key1	value1
1		
2	key2	value2
3		
4		
5	key4	value4

bucket

将一个键值对放进字典的底层过程

```
1 a = {}
2 a["name"]="gaoqi"
```

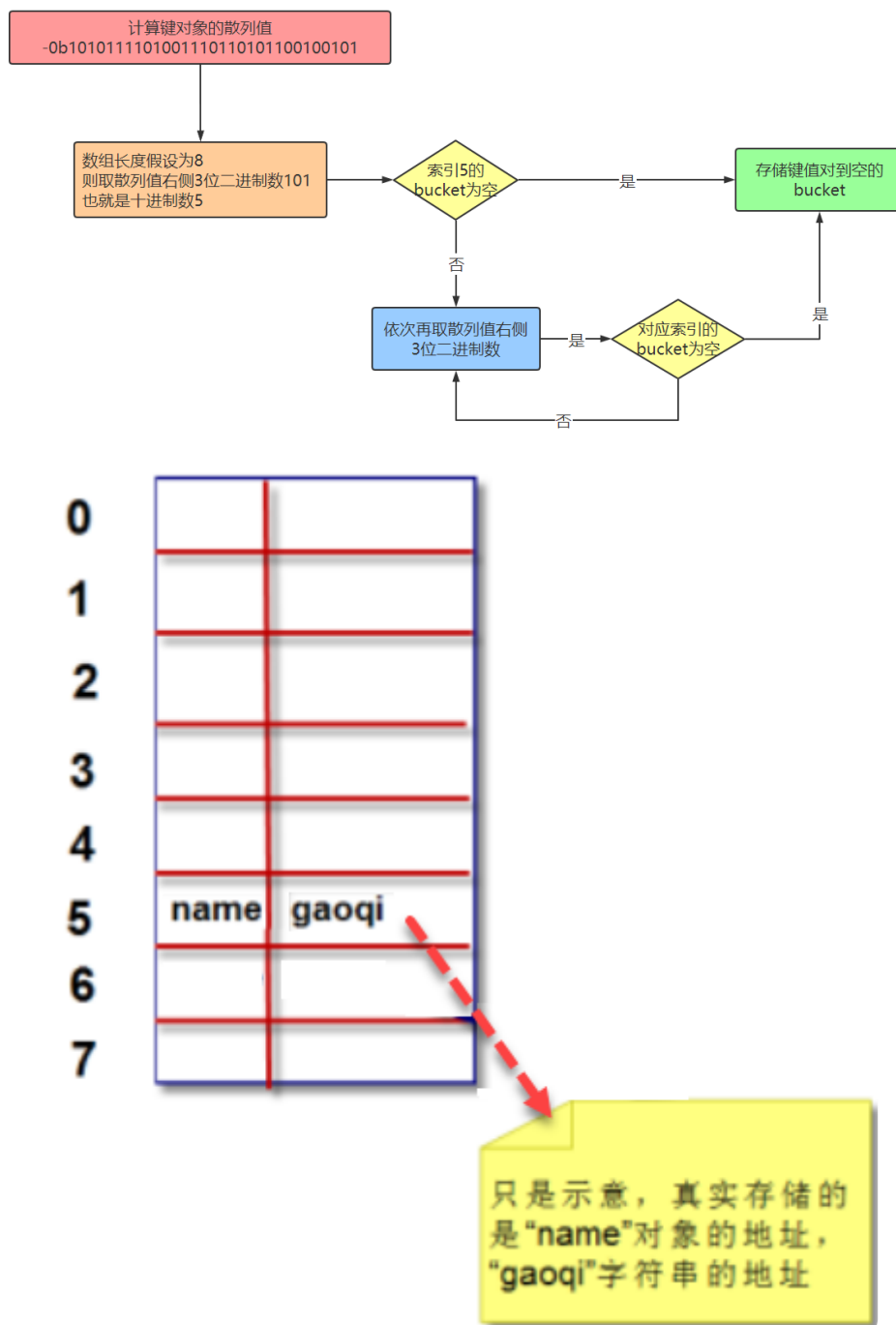
假设字典a对象创建完后，数组长度为8：

0	
1	
2	
3	
4	
5	
6	
7	

我们要把“name”=“gaoqi”这个键值对放到字典对象a中，首先第一步需要计算键“name”的散列值。Python中可以通过hash()来计算。

```
1 >>> bin(hash("name"))
2 '-0b1010111101001110110101100100101'
```

由于数组长度为8，我们可以拿计算出的散列值的最右边3位数字作为偏移量，即“101”，十进制是数字5。我们查看偏移量5，对应的bucket是否为空。如果为空，则将键值对放进去。如果不为空，则依次取右边3位作为偏移量，即“100”，十进制是数字4。再查看偏移量为4的bucket是否为空。直到找到为空的bucket将键值对放进去。流程图如下：



扩容

- python会根据散列表的拥挤程度扩容。“扩容”指的是:创造更大的数组，将原有内容拷贝到新数组中。
- 接近2/3时，数组就会扩容。

根据键查找“键值对”的底层过程

明白了，一个键值对是如何存储到数组中的，根据键对象取到值对象，理解起来就简单了。

```

1 >>> a.get("name")
2 'gaoqi'

```

当调用`a.get("name")`，就是根据键“name”查找到“键值对”，从而找到值对象“gaoqi”。

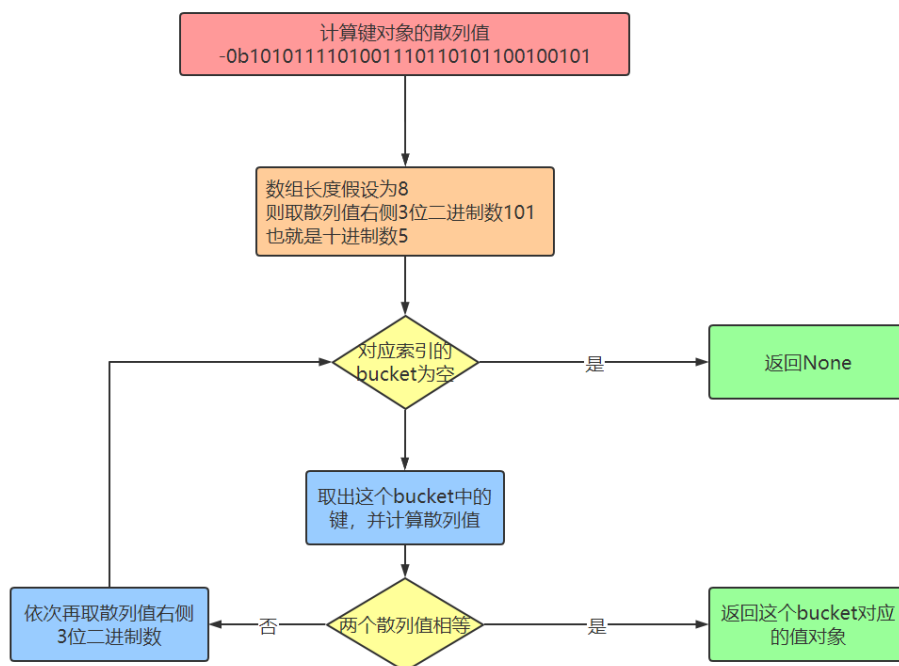
我们仍然要首先计算“name”对象的散列值：

```

1 >>> bin(hash("name"))
2 '-0b1010111101001110110101100100101'

```

和存储的底层流程算法一致，也是依次取散列值的不同位置的数字。假设数组长度为8，我们可以拿计算出的散列值的最右边3位数字作为偏移量，即 `101`，十进制是数字5。我们查看偏移量5，对应的 `bucket` 是否为空。如果为空，则返回 `None`。如果不为空，则将这个 `bucket` 的键对象计算对应散列值，和我们的散列值进行比较，如果相等。则对应“值对象”返回。如果不相等，则再依次取其他几位数字，重新计算偏移量。依次取完后，仍然没有找到。则返回 `None`。流程图如下：



用法总结：

- ① 字典在内存中开销巨大，典型的空间换时间。
- ② 键查询速度很快
- ③ 往字典里面添加新键值对可能导致扩容，导致散列表中键的次序变化。因此，不要在遍历字典的同时进行字典的修改
- ④ 键必须可散列
 - ① 数字、字符串、元组，都是可散列的
 - ② 自定义对象需要支持下面三点:(面向对象章节中再展开说)
 - ① 支持 `hash()` 函数
 - ② 支持通过 `__eq__()` 方法检测相等性
 - ③ 若 `a==b` 为真，则 `hash(a)==hash(b)` 也为真

集合

我的集合，不论资排辈，
不重复设岗！



集合是无序可变，元素不能重复。实际上，集合底层是字典实现，集合的所有元素都是字典中的“键对象”，因此是不能重复的且唯一的。

集合创建和删除

- 1 使用{}创建集合对象，并使用add()方法添加元素

```
1 a = {3, 5, 7}
2 a.add(9)      #{9, 3, 5, 7}
```

- 2 使用set()，将列表、元组等可迭代对象转成集合。如果原来数据存在重复数据，则只保留一个

```
1 a = ['a', 'b', 'c', 'b']
2 b = set(a)  #{'b', 'a', 'c'}
```

- 3 remove()删除指定元素；clear()清空整个集合

```
1 a = {10, 20, 30, 40, 50}
2 a.remove(20)    #{10, 50, 40, 30}
```

集合相关操作

像数学中概念一样，Python对集合也提供了并集、交集、差集等运算。我们给出示例：

```
1 >>> a = {1, 3, 'sxt'}
2 >>> b = {'he', 'it', 'sxt'}
3 >>> a|b                                #并集
4 {1, 3, 'sxt', 'he', 'it'}
5 >>> a&b                                #交集
6 {'sxt'}
7 >>> a-b                                #差集
```

```

8  {1, 3}
9  >>> a.union(b)                #并集
10 {1, 3, 'sxt', 'he', 'it'}
11 >>> a.intersection(b)         #交集
12 {'sxt'}
13 >>> a.difference(b)           #差集
14 {1, 3}

```

实时效果反馈

1. 关于集合，如下说法正确的是：

- ☒ A 集合是有序可变，元素不能重复
- ☒ B 集合底层是字典，集合的所有元素都是字典中的“键对象”，因此不能重复
- ☐ C 集合是无序可变，元素可以重复
- ☐ D 集合底层是列表

答案

1=>B

章节实操作业

- ① 画出代码 `a = [100,200,300]` 的内存存储示意图。
- ② 使用 `range` 生成序列： `30,40,50,60,70,80`
- ③ 推导式生成列表： `a = [x*2 for x in range(100) if x%9==0]`，手写出结果
- ④ 使用二维列表存储表格信息，并画出简单的内存存储示意图

姓名	年龄	薪资	城市
高小一	18	30000	北京
高小二	19	20000	上海
高小五	20	10000	深圳

- 5 元组和列表有哪些共同点？有哪些不同点？
- 6 创建一个字典对象，包含如下信息：
支出金额：300.15，支出日期：2018.10.18，支出人：高小七
- 7 使用字典存储行数据，最后将整个表使用列表存储起来。

姓名	年龄	薪资	城市
高小一	18	30000	北京
高小二	19	20000	上海
高小五	20	10000	深圳

- 8 用文字和自己画的示意图描述：字典存储一个键值对的底层过程。
- 9 集合和字典有什么关系？